TRƯỜNG ĐẠI HỌC
SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
HCMC University of Technology and Education
HCMUTE

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

CÔNG NGHỆ PHẦN MỀM MỚI (.....)

Express Framework

THS. NGUYỄN HỮU TRUNG

PR33
WEB
0479

Thông tin Giảng viên

- Ths. Nguyễn Hữu Trung
- Khoa Công Nghệ Thông Tin
- Trường Đại học Sư Phạm Kỹ Thuật TP.HCM
- 090.861.7108
- trungnh@hcmute.edu.vn
- https://www.youtube.com/@baigiai

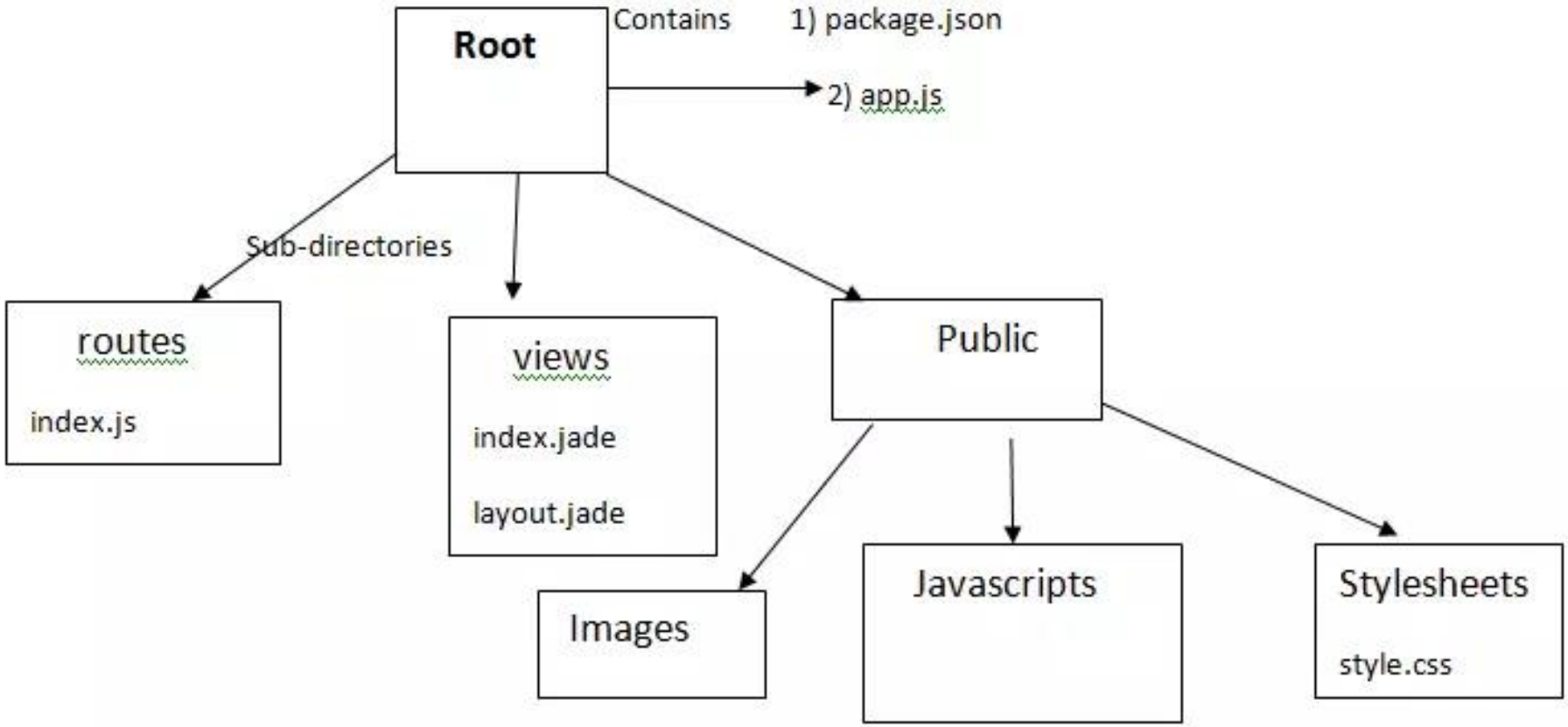
Nội dung bài học

- ExpressJS là gì
- Cấu trúc của ExpressJS
- Cài đặt ExpressJS
- Request & response trong Expresss
- Router trong Express
- File tĩnh trong Express
- Pug viewEngine
- Query Parametter
- Mô hình MVC với ExpressJS

Express Framework là gì

- Express là một web application framework for nodejs, nó cung cấp cho chúng những rất nhiều tính năng mạnh mẽ trên nền tảng web. Express hỗ trợ các phương thức HTTP và middleware tạo ra 1 API rất mạnh mẽ và sử dụng dễ dàng hơn.
- Các tính năng của Express framework:
  - Cho phép thiết lập các lớp trung gian để trả về các HTTP request.
  - Định nghĩa routing có thể được sử dụng với các hành động khác nhau dựa trên phương thức HTTP và URL.
  - Cho phép trả về các trang HTML dựa vào các tham số truyền vào đến template.

Cấu trúc của Express



Cài đặt Express Framework

npm install express –save

- Ngoài ra bạn cũng nên cài các module quan trọng đi cùng với express như:
  - **body-parser** - Đây là một lớp trung gian node.js để xử lí JSON, dự liệu thô, text và mã hóa URL.
  - **cookie-parser**- Chuyển đổi header của Cookie và phân bố đến các req.cookies
  - **multer** - Đây là một thành phần trung gian trong node.js để xử lí phần multipart/form-data.

```
npm install body-parser --save
npm install cookie-parser --save
npm install multer --save
```

Ví dụ: Hello world


```
var express = require('express');
var app = express();
app.get('/', function (req, res) {
   res.send('Hello World');
});
var server = app.listen(3000, function () {
  var host = server.address().address
  var port = server.address().port
  console.log("Ung dung Node.js dang hoat dong
tai dia chi: http://%s:%s", host, port)
});
```

Tạo file server.js

Chạy node server.js

Request & response trong Expresss

- Express sử dụng một hàm callback có các tham số là các đối tượng request và response
  - Request - Biểu diễn một HTTP request và có các thuộc tính cho các request như các chuỗi truy vấn, tham số, body, HTTP header và những phần khác.
  - Response - Biểu diễn một HTTP response được ứng dụng Express gửi đi khi nó nhận về một HTTP request.

```
app.get('/', function (req, res) {
   //code here
});
```

Router trong Express

- Router là một Object (khác Routing nhé), nó là một instance riêng của middleware và routes. Chính vì nó là một instance của middleware và route nên nó có các chức năng của cả hai. Chúng ta có thể gọi nó là một *mini-application*.

```
// invoked for any requests passed to this router
router.use(function(req, res, next) {
  // .. some logic here .. like any other middleware
  next();
});

// will handle any request that ends in /events
// depends on where the router is "use()'d"
router.get('/events', function(req, res, next) {
  // ..
});
```

Chúng ta cũng có thể sử dụng Router để chia route

```
app.use('/calendar', router);
```

- **Tìm hiểu các method all của router**
  - router.all(). Method này phù hợp với việc định nghĩa mang tính chất toàn cục cho các prefix

```
router.all('*', requireAuthentication, loadUser);
```

```
router.all('*', requireAuthentication)
router.all('*', loadUser);
```

ta có một prefix đã được xác định là /api/ thay vì dùng * . Nghĩa là trước khi request vào các route bên trong API thì phải qua một thao tác xác thực requireAuthentication

```
router.all('/api/*', requireAuthentication);
```

Tìm hiểu router.METHOD()

•Router.METHOD() cung cấp cho chúng ta chức năng Routing trong ExpressJS. Cụ thể METHOD() ở đây là các HTTP method mà chúng ta thường xuyên sử dụng. Chẳng hạn GET, POST, PUT,...
•Lưu ý là tên method phải được viết thường nhé (lowercase)

```
router.get('/user/profile', function(req, res,
next) { res.send('user profile'); });
```

```
router.post('/'update/user/:id', function (res,
req, next) { res.send('Update user'); });
```

```
router.put('/update/posts/:id', function (req,
res, next) { res.send('Update post'); });
```

Nếu các bạn muốn bảo mật hơn thì có thể sử dụng Regex để bắt các Endpoint.

```
router.get(/^\/commits\/(\w+)(?:\.\.(\w+))?$/, function(req, res){ var from =
req.params[0]; var to = req.params[1] || 'HEAD'; res.send('commit range ' + from +
'..' + to); });
```

Router trong Express

- Ví dụ:

```
var express = require('express');
var app = express();
// Phuong thuc get() phan hoi mot GET Request
app.get('/', function (req, res) {
   console.log("GET Request");
   res.send('Hello GET');
});
// Phuong thuc post() phan hoi mot POST Request
app.post('/', function (req, res) {
   console.log("POST Request");
   res.send('Hello POST');
});
// Phuong thuc delete() phan hoi mot DELETE Request.
app.delete('/delete', function (req, res) {
   console.log("DELETE Request");
   res.send('Hello DELETE');
});
```

Router trong Express

- Ví dụ

```
// Phuong thuc nay phan hoi mot GET Request có dạng
abcd, abxcd, ab123cd, ...
app.get('/ab*cd', function(req, res) {
   console.log("GET request /ab*cd");
   res.send('Page Pattern Match');
});

var server = app.listen(3000, function () {
  var host = server.address().address
  var port = server.address().port

  console.log("Ung dung Node.js dang hoat dong tai
dia chi: http://%s:%s", host, port);
});
```

File tĩnh trong Express

- Express cung cấp tiện ích **express.static** để phục vụ cho các file tĩnh như hình ảnh, css, js, ...Về cơ bản, bạn chỉ cần truyền tên thư mục nơi bạn giữ các file này, express.static sẽ sử dụng file đó một cách trực tiếp. Giả sử ứng dụng của bạn có cấu trúc như sau:

```
node_modules
server.js
public/
public/images
public/images/logo.png
```

```
app.use(express.static('public'));
```

Ví dụ: Hello world

- Bước 1: Cài đặt Nodejs mới nhất trên trang chủ NodeJS
- Bước 2: Khởi động VSCode và Tạo 1 thư mục mới trên máy bạn đặt tên là: Helloworld
- Bước 3: Tạo file package.json: (xem lại cách tạo ở bài trước)

```
{
  "name": "helloworld",
  "version": "1.0.0",
  "description": "Hello World",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.js"
  },
  "author": "Nguyen Huu Trung",
  "license": "ISC",
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

Ví dụ: Hello world

- Bước 4: Cài đặt Express: npm install --save express@<version>
- Bước 5: Tạo 1 file index.js

```
var express = require('express');
var app = express();
// Phuong thuc get() phan hoi mot GET Request
app.get('/', function (req, res) {
   console.log("GET Request");
   res.send('<h1>Hello world</h1>');
});
var server = app.listen(3000, function () {
  var host = server.address().address
  var port = server.address().port

  console.log("http://%s:%s", host, port);
});
```

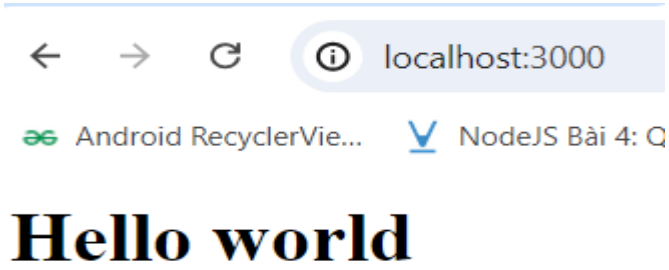
- Trong đó:
  - Khởi tạo **express** với tên app để cung cấp cho **http server**
  - Định nghĩa 1 route handler '/' để xử lý khi có request đến trang web
  - Quy định http server chạy trên cổng 3000

Ví dụ: Hello world

- Bước 6: Chạy thử: vào Terminal của VSCode gõ: node index.js và kết quả:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL CONSOLE

```
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> node index.js
http://:::3000
```



- Với ví dụ trên ta trả về 1 string Hello world, để trả về 1 file html ta sửa lại code như sau: trong đó tạo file index.html

```
app.get('/', function (req, res) {
   res.sendFile('index.html' , { root : __dirname});
});
```

Ví dụ: Hello world

- Trang index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Document</title>
</head>

<body>
    <ul id="messages"></ul>
    <form action="">
        <input id="m" autocomplete="off"
/><button>Send</button>
    </form>
</body>
</html>
```

Pug view engine

- Pug là một module giúp ta viết mã HTML một cách cực kì đơn giản
- ***npm install pug –save***
- **app.set('views', './views');** // Thư mục views nằm cùng cấp với file app.js
- **app.set('view engine', 'pug');** // Sử dụng pug làm view engine
- Thêm một file có tên là ***index.pug*** vào thư mục users, chuyển mã HTML dạng pug vào file này

```
.users
 table
  thead
   tr
    th  Name
    th  Email
  tbody
   tr
    td  User1
    td  user1@gmail.com
```

```
app.get('/users', function(req, res){
        res.render('users/index');
})
```

app.js

Pug view engine

- Hãy làm cho web của chúng ta trở nên dynamic hơn
- var users = [{name: "User1", email: "user1@gmail.com"}, {name: "User2", email: "user2@gmail.com"}]

```
.users
  table
    thead
      tr
        th  Name
        th  Email
    tbody
    each user in users
      tr
        td=  user.name
        td=  user.email
```

app.js

```
app.get('/users', function(req, res){
    var users = [{name: "User1", email:
"user1@gmail.com"}, {name: "User2",
email: "user2@gmail.com"}]
res.render('users/index',{users: users});
})
```

Query Parameters

- Các chuỗi truy vấn có thể được đính kèm trong một URL. Trong URL, các truy vấn sẽ bắt đầu từ sau dấu ***?*** , mỗi truy vấn là một cặp {key:value}, các cặp ngăn cách nhau bởi kí tự ***&*** tạo thành một object gọi là param và được gửi lên server.
- Với URL: **http://iotstar.vn/users?name=trung&age=40** sẽ được chuyển thành object là:

```
{
    name: “trung",
    age: “40"
}
```

Query Parameters

- Xây dựng chức năng tìm kiếm user theo tên:
  - Bước 1: tạo một route để có thể tìm kiếm user, ở đây mình sẽ tạo một route tại /users/search

```
app.get('/users/search', (req,res) => {
        console.log(req.query);
})
```

  - Bước 2: Tạo một form HTML để search, mình sẽ tạo tại trang index của users

```
form.seach-form(action="/users/search")
  input(name="name", type="text", placeholder="name")
  input(type="submit")
```

Query Parameters


```
const express = require('express'); // Require module express vào project
const app = express(); // Tạo một app mới
const port = 8080; // Định nghĩa cổng để chạy ứng dụng NodeJS của bạn trên server
app.set('views', './views'); // Thư mục views nằm cùng cấp với file app.js
app.set('view engine', 'pug'); // Sử dụng pug làm view engine
var users = [
   {name: "User1", email: "user1@gmail.com"},
   {name: "User2", email: "user2@gmail.com"} ];
app.get('/', function(req, res){
   res.send("<h2>This is my first app</h2>");
})
app.get('/users', function(req, res){
   res.render('users/index',{
      users: users
   });})
```

app.js

Query Parameters


```
app.get('/users/search', (req,res) => {
   var name_search = req.query.name // lấy giá trị của key name trong query parameters gửi lên
   var result = users.filter( (user) => {
      // tìm kiếm chuỗi name_search trong user name.
      // Lưu ý: Chuyển tên về cùng in thường hoặc cùng in hoa để không phân biệt hoa, thường khi tìm kiếm
      return user.name.toLowerCase().indexOf(name_search.toLowerCase()) !== -1
   })
   res.render('users/index', {
      users: result // render lại trang users/index với biến users bây giờ chỉ bao gồm các kết quả phù hợp
   });
})
app.listen(port, function(){
   console.log('Your app running on port '+ port);
})
```

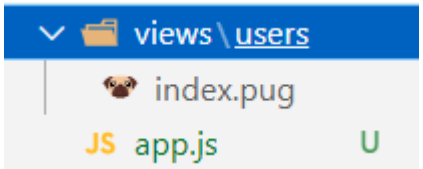
app.js

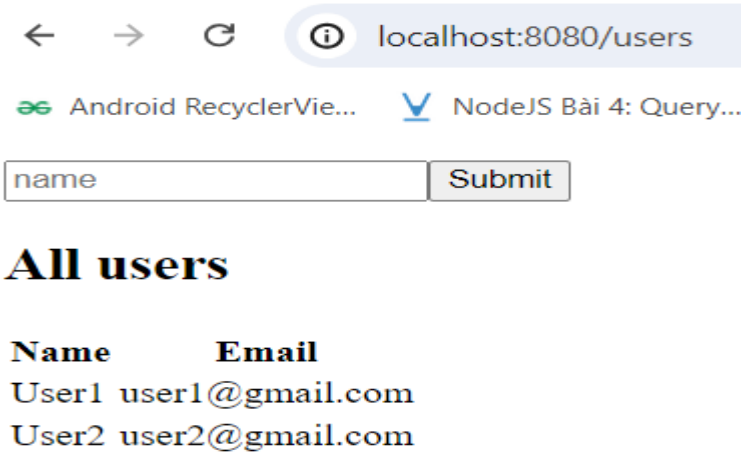
Query Parameters

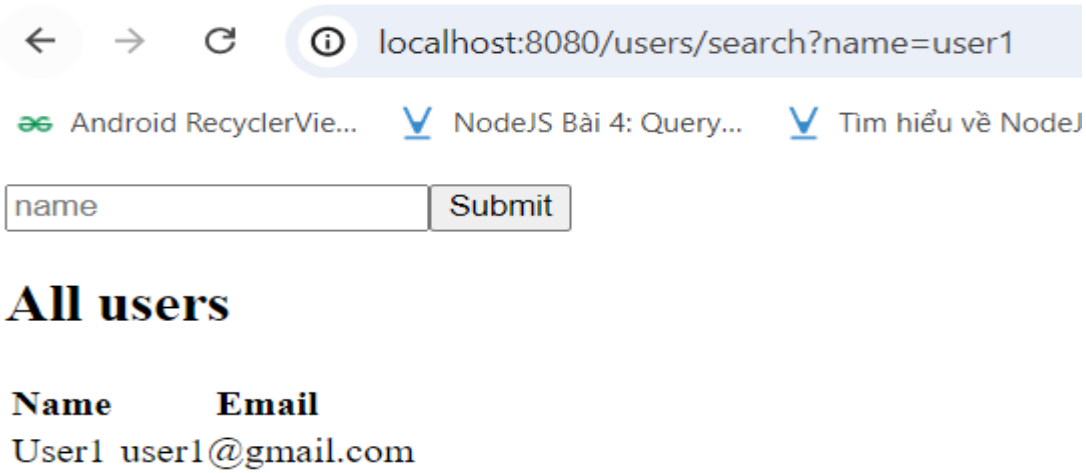
Index.pug

```
form.seach-form(action="/users/search")
  input(name="name", type="text", placeholder="name")
  input(type="submit")

.users
  h2 All users
  if users.length > 0
    table
      thead
        tr
          th  Name
          th  Email
      tbody
        each user in users
          tr
            td=  user.name
            td=  user.email
  else
    p No user to display
```

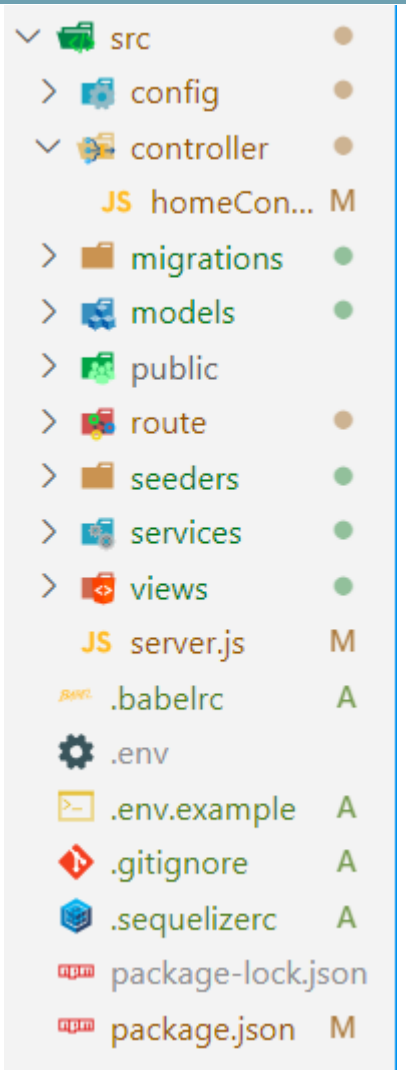






Mô hình MVC trong ExpressJS

- Mô hình MVC (Model – View – Controller) là một kiến trúc phần mềm hay mô hình thiết kế được sử dụng trong kỹ thuật phần mềm. Nó giúp tách ứng dụng thành 3 phần khác nhau: Model (Xử lý dữ liệu), View (Hiển thị) và Controller (Điều khiển). Mỗi thành phần có một nhiệm vụ riêng biệt và độc lập với nhau.
- Với **epress-generator**, chúng ta đã có sẵn 2 thư mục **views** chứa các file template (ví dụ .ejs), các file này được dùng để hiển thị dữ liệu, tương tự với phần Views trong MVC, thư mục **routes** dùng để chuyển hướng các URL đến các hàm xử lý tương ứng, tương tự thành phần controller trong MVC



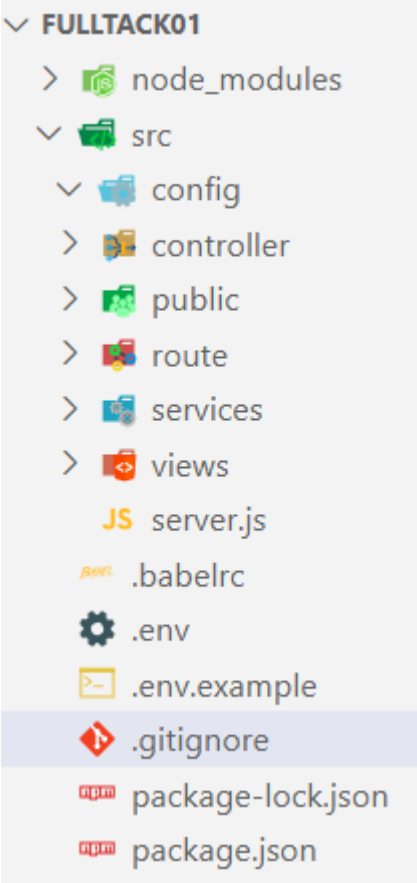

```
$ npm install -g express-generator
$ express --view=ejs //hoặc pug
```

Ví dụ tạo Project bằng mô hình MVC trong ExpressJS

- Bước 1: Tạo project: Khởi động VSCode và tạo thư mục project: FullStack01
- Bước 2: Tạo file **package.json** quản lý project, thư viện bằng lệnh **npm init** trong **terminal -> nhấn enter**
- Bước 3: Cài đặt Express: **npm install express**
- Bước 4:
  - Cài body-parser để truyền tham số từ client lên server: **npm install body-parser**
  - Cài dotenv giúp dùng câu lệnh .env lấy tham số chung: **npm install dotenv**
  - Cài ejs giúp làm view engine: **npm install ejs**
  - Cài các package dev: **npm install --save-dev @babel/core @babel/node @babel/preset-env nodemon**
    - Dùng Babel làm compiler
    - Dùng nodemon để tự động restart project khi cập nhật code

Ví dụ tạo Project bằng mô hình MVC trong ExpressJS

- Bước 5: Tạo file **.gitignore** dùng để ngăn việc push lên github



.gitignore

```
/node_modules
/vendor
/.idea
.idea/
.env
package-lock.json
```

.env

```
PORT=8088
```

.babelrc

```
{
    "presets": [
        "@babel/preset-env"
    ]
}
```

.env.example

```
PORT=
```

Ví dụ tạo Project bằng mô hình MVC trong ExpressJS

- Bước 6: Cấu hình view engine: tạo thư mục src\config rồi tạo file viewEngine.js

```
import express from "express";
// cú pháp khác tương đương:
var express = require('express');
//javascript theo ES6
let configViewEngine = (app) => {
    app.use(express.static("./src/public"));//Thiết lập thư
mục tĩnh chứa images, css,..
    app.set("view engine", "ejs"); //thiết lập viewEngine
    app.set("views", "./src/views") //thư mục chứa views
}
module.exports = configViewEngine; //xuất hàm ra
```

Ví dụ tạo Project bằng mô hình MVC trong ExpressJS

- Bước 7: Cấu hình Route: tạo file web.js trong thư mục src\route

```
import express from "express"; //gọi Express
import homeController from "../controller/homeController"; //gọi
controller
let router = express.Router();  //khởi tạo Route

let initWebRoutes = (app) => {
    //cách 1
    router.get('/', (req,res) => {
        return res.send('Nguyễn Hữu Trung');
    });
    //cách 2
    router.get('/home', homeController.getHomePage); //url cho
trang chủ
    router.get('/about', homeController.getAboutPage); //url cho
trang about

    return app.use("/", router); //url mặc định
}
module.exports = initWebRoutes;
```

Ví dụ tạo Project bằng mô hình MVC trong ExpressJS

- Bước 8: Cấu hình server.js

```
import express from "express"; //nạp express
import bodyParser from "body-parser"; //nạp body-parser lấy tham
số từ client /user?id=7
import viewEngine from "./config/viewEngine"; //nạp viewEngine
import initWebRoutes from './route/web';//nạp file web từ Route
require('dotenv').config();
let app = express();
//config app
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }))
viewEngine(app);
initWebRoutes(app);
let port = process.env.PORT || 6969; //tạo tham số port lấy từ
.env
//Port === undefined => port = 6969 //chạy server
app.listen(port, () => {
    //callback
   console.log("Backend Nodejs is runing on the port : " + port)
})
```

Ví dụ tạo Project bằng mô hình MVC trong ExpressJS

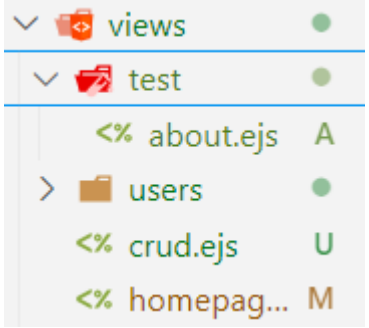
- Bước 9: Cấu hình lại file **package.json** để chạy server

```
{
  "name": "fullstack01",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "nodemon --exec babel-node src/server.js"
  },
  "author": "Nguyen Huu Trung",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "body-parser": "^1.20.2","dotenv": "^16.4.5","ejs": "^3.1.10",
    "express": "^4.19.2"
  },
  "devDependencies": {
    "@babel/core": "^7.24.7", "@babel/node": "^7.24.7",
    "@babel/preset-env": "^7.24.7", "nodemon": "^3.1.4"
  }
}
```

Ví dụ tạo Project bằng mô hình MVC trong ExpressJS

- Bước 10: Tạo thư mục src\Controllers để tạo homeController.js

```
//hàm getHomePage
let getHomePage = (req, res) => {
    //return res.send('Nguyễn Hữu Trung');
    return res.render('homepage.ejs');//trả về view
}
//hàm getAbout
let getAboutPage = (req, res) => {
    return res.render('test/about.ejs');
}
// object: {
//     key: '',
//     value: ''
// }
//export ra object
module.exports = {
    getHomePage: getHomePage,
    getAboutPage: getAboutPage
}
```



Ví dụ tạo Project bằng mô hình MVC trong ExpressJS

- Bước 11: Tạo homepage.ejs trong thư mục src\views

JS homeController.js × | <% about.ejs | <% homepage.ejs ×

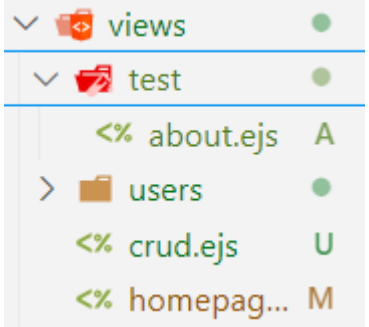
src > views > <% homepage.ejs > html > body > h1

```html
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>

<body>
    <h1>Hello world Nguyễn Hữu Trung</h1>
</body>

</html>
```



Layout web với ViewEngine EJS

- Cài đặt thư viện ejs: **npm install ejs –save**
- Mở file app.js lên và sửa thành **app.set('view engine', 'ejs');**
- Xây dựng partials common layout với ejs
  - Tạo các file **header.ejs, footer.ejs, sidebar.ejs** trong views
  - Sử dụng ejs partials common layout: Cú pháp để sử dụng ejs partials : **<% include FILENAME%>**
  - Mở file index.ejs lên và thêm vào đúng vị trí cần:

```
<% include header%>

<% include sidebar%>

<% include footer%>
```